

Tech Digest Bot v3.0 — Documentación Completa

Índice

1. Descripción del Proyecto
 2. Arquitectura General
 3. Flujo de Datos Completo
 4. Qué se hizo para construirlo
 5. Estructura de Ficheros
 6. Subagentes de IA
 7. Documentación del Código — bot_v2.py
 8. Documentación del Código — telegram_bot.py (Panel)
 9. Documentación del Código — sources.yaml
 10. Documentación del Código — GitHub Actions
 11. Variables de Entorno y Secrets
 12. Costes del Sistema
 13. Instrucciones de Uso
-

1. Descripción del Proyecto

Tech Digest Bot v3.0 es un sistema automatizado de agregación, filtrado y publicación de noticias tecnológicas en un canal de Telegram.

El bot recopila artículos de **13 fuentes RSS** y **13 canales de YouTube** cada hora, los procesa con inteligencia artificial gratuita (Groq/LLaMA), selecciona las 6 mejores noticias, las traduce al español y las publica automáticamente en el canal "**Últimas Novedades**" con imagen y formato profesional.

Características principales

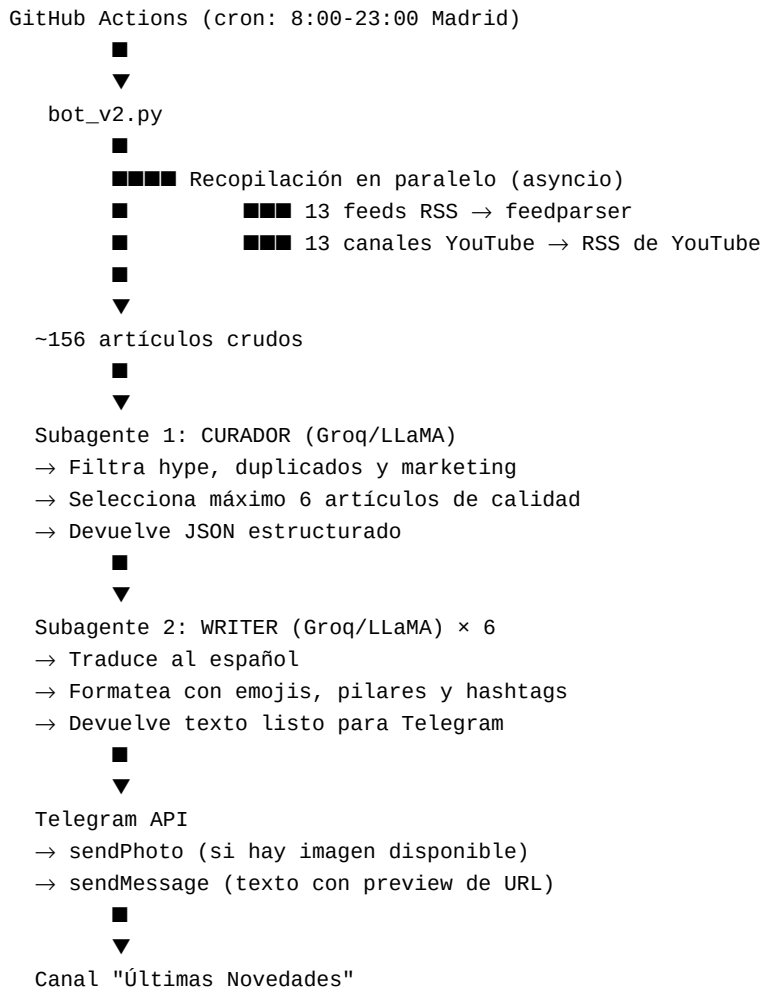
- **Completamente gratuito** — usa Groq API (gratis), GitHub Actions (gratis) y Telegram Bot API (gratis)
- **Totalmente automático** — funciona 24/7 sin intervención humana
- **Bilingüe inteligente** — recopila en inglés y español, publica siempre en español
- **Visual** — cada noticia incluye imagen real del artículo o thumbnail de YouTube
- **Pipeline de subagentes** — dos IAs especializadas: una filtra, otra redacta

- **Panel de control** — interfaz de botones en Telegram para gestión manual

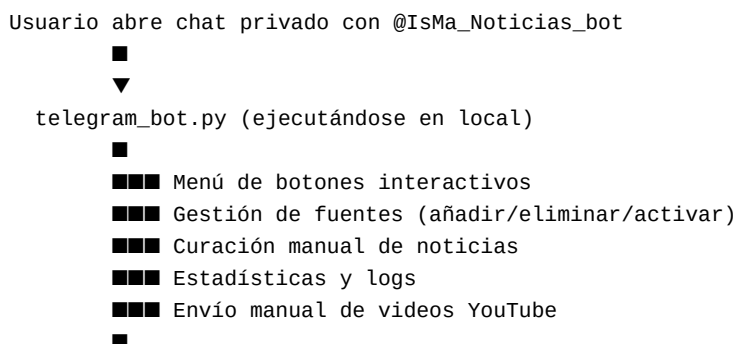
2. Arquitectura General

El sistema tiene dos modos de operación:

Modo Automático (GitHub Actions, cada hora)



Modo Manual (Panel de Control)



3. Flujo de Datos Completo

Paso 1 — Carga de configuración

`bot_v2.py` lee `sources.yaml` y obtiene la lista de 26 fuentes (13 RSS + 13 YouTube).

Paso 2 — Recopilación paralela

Lanza todas las peticiones HTTP simultáneamente con `asyncio.gather()`. Para cada fuente RSS extrae: título, URL, resumen e imagen si está disponible en el feed. Para YouTube genera automáticamente la URL del thumbnail (`img.youtube.com/vi/{id}/maxresdefault.jpg`).

Paso 3 — Curación (Subagente 1)

Construye un mensaje de texto con todos los artículos (limitado a 18.000 caracteres para no superar el límite de Groq) y lo envía al modelo LLaMA 3.3-70B. El modelo devuelve un array JSON con cada artículo marcado como APPROVED o REJECTED.

Paso 4 — Redacción (Subagente 2)

Por cada artículo aprobado (máximo 6), envía el JSON del artículo al mismo modelo con un prompt diferente para que lo redacte en español con formato Telegram: emojis de pilar, negritas, hashtags y URL directa.

Paso 5 — Publicación

Para cada post generado busca si existe imagen asociada en los artículos originales. Si la hay, usa `sendPhoto` con el texto como caption. Si no, usa `sendMessage` con `disable_web_page_preview: False` para que Telegram genere automáticamente una vista previa del enlace.

4. Qué se hizo para construirlo

Fase 1 — Bot original

El proyecto partía de un bot básico que usaba **Groq API** (modelo LLaMA gratuito) para generar un digest de texto plano y lo enviaba como un único mensaje largo a Telegram.

Fase 2 — Migración a Claude API

Se migró de Groq a **Anthropic Claude API** para mejorar la calidad editorial. Se integró el sistema de prompts mejorado con criterios editoriales detallados.

Fase 3 — Reducción de fuentes

Se eliminaron más de 30 fuentes que no aportaban valor (ArXiv, HackerNews, Dev.to, Reddit, y múltiples blogs académicos) para reducir el ruido y mejorar la señal. De 40+ fuentes se pasó a 26 fuentes de alta calidad.

Fase 4 — Formato visual

Se rediseñó el sistema de envío para que cada noticia llegue como un mensaje individual con:

- Imagen real extraída del feed RSS o thumbnail de YouTube
- Título, descripción, fuente y URL directa visible

Fase 5 — Pipeline de subagentes

Se implementó una arquitectura de **dos subagentes especializados** en lugar de un único prompt monolítico. El Curador filtra con criterios estrictos y el Writer redacta con estilo y formato.

Fase 6 — Vuelta a Groq

Para mantener el coste en **\$0**, se migró de nuevo a Groq con el nuevo pipeline de subagentes. Se añadió manejo automático de rate limit con reintentos.

Fase 7 — Traducción al español

Se configuró el Writer para traducir siempre al español, manteniendo los nombres de tecnologías en su idioma original.

Fase 8 — Panel de control

Se habilitó `telegram_bot.py`, un bot de Telegram con **menús interactivos de botones** que permite gestionar todo el sistema desde el móvil sin necesidad de editar código.

Fase 9 — Horario optimizado

Se configuró el cron de GitHub Actions para ejecutar solo entre las 8:00 y las 23:00 hora Madrid, evitando notificaciones nocturnas.

5. Estructura de Ficheros

Bot_News/

■		
■■■	bot_v2.py	← Motor principal del bot automático
■■■	telegram_bot.py	← Panel de control interactivo
■■■	sources.yaml	← Lista de fuentes RSS y YouTube
■■■	db_manager.py	← Gestor de base de datos Supabase
■■■	config.py	← Configuración de modelos IA y categorías
■■■	requirements.txt	← Dependencias Python
■■■	.env	← Variables de entorno (local, no en git)
■		
■■■	.github/	
■	■■■ workflows/	
■	■■■ daily.yml	← Automatización GitHub Actions
■		
■■■	.claude/	
	■■■ agents/	
	■■■ content-curator-telegram.md	← Definición Subagente Curador
	■■■ telegram-content-writer.md	← Definición Subagente Writer
	■■■ tech-translator-es.md	← Definición Subagente Traductor
	■■■ weekly-digest-curator.md	← Definición Subagente Resumen Semanal

Descripción de cada fichero

Fichero	Función
<code>`bot_v2.py`</code>	Núcleo del sistema. Recopila noticias, ejecuta el pipeline de subagentes y publica en Telegram. Se ejecuta automáticamente cada hora vía GitHub Actions.
<code>`telegram_bot.py`</code>	Bot de Telegram con interfaz de botones para gestionar el sistema desde el móvil. Necesita estar ejecutándose localmente para funcionar.
<code>`sources.yaml`</code>	Archivo de configuración declarativo con todas las fuentes. Añadir o quitar fuentes sin tocar código Python.
<code>`db_manager.py`</code>	Capa de abstracción para la base de datos Supabase. Gestiona fuentes, noticias, configuración y logs.
<code>`config.py`</code>	Constantes de configuración: modelos de IA disponibles, categorías de noticias y prompt del sistema por defecto.
<code>`.env`</code>	Variables de entorno sensibles (API keys, tokens). Solo existe localmente, nunca se sube a GitHub.
<code>`daily.yml`</code>	Workflow de GitHub Actions que ejecuta <code>`bot_v2.py`</code> cada hora de 8:00 a 23:00 hora Madrid.

6. Subagentes de IA

El sistema define cuatro subagentes especializados. Dos se usan en el pipeline automático; los otros dos están disponibles para uso manual desde Claude Code.

Subagente 1 — ``content-curator-telegram``

Rol: Curador editorial implacable. Actúa como portero del canal.

Función en el pipeline: Recibe los ~156 artículos crudos recopilados de todas las fuentes y los filtra con criterios editoriales estrictos. Solo aprueba contenido con valor técnico real.

Criterios de rechazo:

- Marketing corporativo disfrazado de noticia
- Clickbait sin benchmarks verificables
- Noticias duplicadas (elige la fuente más autorizada)
- Opiniones vagas sin datos concretos
- Artículos con más de una semana de antigüedad

Criterios de aprobación:

- Contenido técnico con profundidad real
- Lanzamientos de software, frameworks o modelos de IA
- Vulnerabilidades de seguridad (CVE)
- Tutoriales con novedad técnica
- Videos educativos de canales especializados

Formato de salida: Array JSON con cada artículo marcado como APPROVED o REJECTED, incluyendo el pilar temático (CODE, AI, INFRA, INNOVATION), tags técnicos y un resumen de dos frases.

Resultado típico: De 156 artículos crudos → 6 artículos aprobados de alta calidad.

Subagente 2 — `telegram-content-writer`

Rol: Redactor senior de contenido técnico en español.

Función en el pipeline: Recibe el JSON de cada artículo aprobado por el Curador y lo transforma en un post listo para publicar en Telegram.

Reglas de formato:

- **Cabecera:** emoji del pilar + nombre del pilar en mayúsculas + título en negrita
- **■ PROGRAMACIÓN:** *Título*
- **■ INTELIGENCIA ARTIFICIAL:** *Título*
- **■■ IT & INFRAESTRUCTURA:** *Título*
- **■ INNOVACIÓN TECH:** *Título*
- **Cuerpo:** 2 párrafos cortos con los 3 conceptos técnicos clave en negrita
- **URL:** dirección directa visible (no hipervínculo) para que Telegram genere la vista previa
- **Hashtags:** 3-4 etiquetas técnicas al final

Reglas de escritura:

- Todo en español (nombres de tecnologías se mantienen en inglés)
 - Máximo 900 caracteres por post
 - Sin presentaciones ni explicaciones — solo el post
-

Subagente 3 — *`tech-translator-es` *(uso manual)**

Rol: Traductor técnico inglés → español.

Función: Traduce artículos, documentación o JSONs técnicos del inglés al español preservando la terminología técnica correcta. Diseñado para audiencias de desarrolladores hispanohablantes.

Uso: Se invoca manualmente desde Claude Code cuando hay contenido en inglés que necesita traducción antes de publicar.

Subagente 4 — *`weekly-digest-curator` *(uso manual)**

Rol: Editor del resumen semanal.

Función: Analiza todos los artículos publicados durante la semana y genera un único post de resumen ultra-compacto con los 5 impactos más importantes. Diseñado para publicar los domingos.

Uso: Se invoca manualmente desde Claude Code los domingos con el historial de la semana.

7. Documentación del Código — *bot_v2.py*

Bloque de importaciones

```
import os          # Acceso a variables de entorno del sistema
import logging     # Sistema de logs con niveles (INFO, WARNING, ERROR)
import asyncio     # Programación asíncrona para peticiones HTTP paralelas
import re          # Expresiones regulares para extraer video IDs de YouTube
import json        # Parseo del JSON que devuelve Groq

import httpx       # Cliente HTTP asíncrono (reemplaza a requests)
import feedparser  # Parser de feeds RSS y Atom
import yaml        # Lector de archivos YAML (sources.yaml)
```

Función `load_env_file()`

```
def load_env_file():
    env_path = ".env"
    # Solo carga el .env si existe (en local). En GitHub Actions
    # las variables vienen de los Secrets, no del .env
    if os.path.exists(env_path):
        with open(env_path, "r", encoding="utf-8") as f:
            for line in f:
                line = line.strip()
                # Ignora líneas vacías y comentarios (#)
                if line and not line.startswith("#") and "=" in line:
                    key, value = line.split("=", 1)
                    # setdefault: no sobrescribe si ya está en el entorno
                    os.environ.setdefault(key.strip(), value.strip())
```

Propósito: Permite ejecutar el bot localmente con un archivo `.env` sin necesidad de instalar `python-dotenv`. En producción (GitHub Actions), las variables ya están en el entorno.

Constantes de configuración

```
GROQ_MODEL = "llama-3.3-70b-versatile" # Modelo LLaMA de 70B parámetros (gratuito)
MAX_ITEMS_PER_SOURCE = 10 # Máximo artículos a recoger por fuente
MAX_INPUT_CHARS = 18_000 # Límite de caracteres enviados a Groq (evita error 413)
REQUEST_TIMEOUT = 15 # Segundos de espera para peticiones HTTP
SOURCES_FILE = "sources.yaml" # Fichero de configuración de fuentes
```

Función `load_sources(filename)`

```
def load_sources(filename: str) -> dict:
    try:
        with open(filename, "r", encoding="utf-8") as f:
            return yaml.safe_load(f) or {} # safe_load evita ejecución de código YAML
    except FileNotFoundError:
        log.warning(f"{filename} no encontrado.")
        return {} # Devuelve dict vacío si no existe el fichero
```

Propósito: Lee `sources.yaml` y devuelve un diccionario con todas las fuentes configuradas. Usa `safe_load` (no `load`) por seguridad.

Función `fetch_rss(client, name, url)`

```
async def fetch_rss(client, name, url):
    try:
        # GET asíncrono con redirecciones automáticas (muchos feeds redirigen)
        r = await client.get(url, timeout=REQUEST_TIMEOUT, follow_redirects=True)
        r.raise_for_status() # Lanza excepción si HTTP 4xx o 5xx
        feed = feedparser.parse(r.text) # Parsea el XML del feed
        items = []
        for entry in feed.entries[:MAX_ITEMS_PER_SOURCE]: # Solo los 10 primeros
            image_url = None
            # Intenta extraer imagen del campo media:content del feed
            if hasattr(entry, 'media_content') and entry.media_content:
                image_url = entry.media_content[0].get('url')
            items.append({
                "source": name,
                "title": entry.get("title", "")[:100], # Trunca títulos largos
                "url": entry.get("link", ""),
                "summary": entry.get("summary", "")[:300], # Trunca resúmenes
                "image_url": image_url, # None si el feed no incluye imagen
            })
        return items
    except Exception as e:
        log.warning(f"RSS [{name}] falló: {e}")
        return [] # Si una fuente falla, el resto sigue funcionando
```


Propósito: Descarga y parsea un feed RSS. Extrae imagen si está disponible en el feed (campo `media:content`). Si la fuente falla, devuelve lista vacía sin interrumpir el proceso.

Función `fetch_youtube_channels(client, channels)`

```
async def fetch_youtube_channels(client, channels):
    items = []
    for ch in channels:
        if "channel_id" not in ch:
            continue # Salta canales sin channel_id configurado
        # YouTube expone un feed RSS público para cada canal
        url = f"https://www.youtube.com/feeds/videos.xml?channel_id={ch['channel_id']}"
        result = await fetch_rss(client, f"YouTube: {ch['name']}", url)
        for item in result[:3]: # Solo los 3 últimos vídeos por canal
            # Extrae el ID del vídeo de la URL (ej: ?v=dQw4w9WgXcQ)
            match = re.search(r'v=([a-zA-Z0-9_-]+)', item['url'])
            if match:
                # Genera URL del thumbnail en máxima resolución
                item['image_url'] = f"https://img.youtube.com/vi/{match.group(1)}/maxresdefault.jpg"
            items.append(item)
    return items
```

Propósito: Obtiene los últimos vídeos de canales YouTube usando el feed RSS público (no necesita API key de YouTube). Genera automáticamente la URL del thumbnail en alta resolución.

Función `collect_all_news(sources)`

```
async def collect_all_news(sources):
    headers = {"User-Agent": "TechDigestBot/3.0", "Accept": "application/rss+xml, */*"}
    async with httpx.AsyncClient(headers=headers, follow_redirects=True) as client:
        # Crea una tarea asíncrona por cada fuente RSS
        tasks = [fetch_rss(client, f["name"], f["url"]) for f in sources.get("rss_feeds", [])]
        # Añade las tareas de YouTube como un único bloque
        if sources.get("youtube_channels"):
            tasks.append(fetch_youtube_channels(client, sources["youtube_channels"]))
        # Ejecuta TODAS las tareas en paralelo (reduce tiempo de ~30s a ~2s)
        results = await asyncio.gather(*tasks)
        # Aplana la lista de listas en una única lista de artículos
        all_items = [item for sub in results for item in sub]
    return all_items
```

Propósito: Orquesta la recopilación de todas las fuentes en paralelo. Sin `asyncio.gather()`, las 26 fuentes se descargarían secuencialmente en ~30 segundos. Con él, tarda ~2 segundos.

Función `call_groq(system_prompt, user_message)`

```
async def call_groq(system_prompt: str, user_message: str) -> str:
    for attempt in range(5): # Máximo 5 intentos ante rate limit
        async with httpx.AsyncClient() as client:
            r = await client.post(
                "https://api.groq.com/openai/v1/chat/completions",
```

```

headers={"Authorization": f"Bearer {GROQ_API_KEY}", ...},
json={
    "model": GROQ_MODEL,
    "max_tokens": 4096,
    "messages": [
        {"role": "system", "content": system_prompt}, # Instrucciones del agente
        {"role": "user", "content": user_message},     # Artículos a procesar
    ],
    "temperature": 0.3, # Baja temperatura = respuestas más consistentes
},
timeout=60,
)
if r.status_code == 429: # Rate limit de Groq
    wait = int(r.headers.get("retry-after", 10)) # Groq indica cuánto esperar
    await asyncio.sleep(wait) # Espera el tiempo indicado y reintenta
    continue
r.raise_for_status()
return data["choices"][0]["message"]["content"]
raise RuntimeError("Groq: demasiados reintentos por rate limit")

```

Propósito: Cliente genérico para Groq API compatible con OpenAI. Maneja automáticamente el rate limit del plan gratuito (30 req/min) esperando el tiempo exacto que indica Groq en la cabecera retry-after.

Función `curate\_articles(articles)` — Subagente 1

```

async def curate_articles(articles: list) -> list:
    today = datetime.now(timezone.utc).strftime("%A, %d %B %Y")
    lines = [f"DATE: {today}\n"]
    # Construye el mensaje con todos los artículos en formato texto
    for i, a in enumerate(articles, 1):
        lines.append(f"[{i}] SOURCE: {a['source']}")
        lines.append(f"    TITLE: {a['title']}")
        lines.append(f"    URL: {a['url']}")
        if a.get("summary"):
            lines.append(f"    SUMMARY: {a['summary'][:200]}")
        lines.append("")

    user_msg = "\n".join(lines)
    if len(user_msg) > MAX_INPUT_CHARS:
        user_msg = user_msg[:MAX_INPUT_CHARS] # Trunca si supera el límite de Groq

    raw = await call_groq(CURATOR_PROMPT, user_msg)

    # El modelo devuelve JSON pero a veces con texto antes/después
    # La regex extrae solo el array JSON válido
    match = re.search(r'\[.*\]', raw, re.DOTALL)
    if match:
        curated = json.loads(match.group(0))

    # Filtra solo los aprobados
    approved = [a for a in curated if a.get("status") == "APPROVED"]
    return approved

```

Propósito: Implementa el Subagente Curador. Envía todos los artículos al modelo con el CURATOR_PROMPT y parsea el JSON resultante. La regex de extracción maneja casos donde el modelo incluye texto explicativo antes o después del JSON.

Función `write\_post(article)` — Subagente 2

```
async def write_post(article: dict) -> str:
    # Convierte el dict del artículo a JSON con formato legible
    user_msg = json.dumps(article["data"], ensure_ascii=False, indent=2)
    # ensure_ascii=False preserva caracteres como ñ, á, é, etc.
    post = await call_groq(WRITER_PROMPT, user_msg)
    return post.strip() # Elimina espacios/saltos de línea al inicio y final
```

Propósito: Implementa el Subagente Writer. Envía el JSON de un artículo aprobado al modelo con el WRITER_PROMPT para que lo formatee como post de Telegram en español.

Función `send\_post(client, article, post\_text, all\_articles)`

```
async def send_post(client, article, post_text, all_articles):
    article_url = article["data"].get("url", "")

    # Busca la imagen del artículo original comparando URLs
    image_url = None
    for a in all_articles:
        if a.get("url") == article_url and a.get("image_url"):
            image_url = a["image_url"]
            break

    if image_url:
        # Intenta enviar como foto con el post como caption (máx 1024 chars)
        payload = {"chat_id": TELEGRAM_CHAT_ID, "photo": image_url,
                  "caption": post_text[:1024]}
        r = await client.post(url_photo, json=payload, timeout=15)
        if r.status_code == 200:
            return # Éxito con imagen

    # Fallback: texto con preview automático de URL
    # disable_web_page_preview=False activa la vista previa del enlace
    payload = {"chat_id": TELEGRAM_CHAT_ID, "text": post_text,
              "disable_web_page_preview": False}
    await client.post(url_message, json=payload, timeout=15)
```

Propósito: Publica cada noticia en Telegram. Prioriza el envío con imagen (sendPhoto). Si la imagen no está disponible o falla, usa sendMessage con vista previa automática del enlace activada.

Función `main()`

```
async def main():
    # 1. Verifica que las credenciales estén configuradas
    if not all([GROQ_API_KEY, TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID]):
        log.error("Faltan variables de entorno")
        return

    # 2. Carga fuentes y recopila artículos en paralelo
    sources = load_sources(SOURCES_FILE)
```

```

all_articles = await collect_all_news(sources)

# 3. Subagente 1: filtra y selecciona las mejores noticias
curated = await curate_articles(all_articles)

# 4. Para cada artículo aprobado: redacta y publica
async with httpx.AsyncClient() as client:
    for article in curated:
        post_text = await write_post(article)          # Subagente 2
        await send_post(client, article, post_text, all_articles)
        await asyncio.sleep(3.0) # Pausa entre mensajes (evita rate limit de Telegram)

```

Propósito: Punto de entrada que orquesta todo el pipeline en orden secuencial. El `asyncio.sleep(3.0)` entre mensajes respeta el límite de Telegram de ~20 mensajes/minuto en canales.

8. Documentación del Código — telegram_bot.py (Panel)

Arquitectura del panel

El panel es un bot de Telegram con **inline keyboards** (botones interactivos embebidos en los mensajes). Funciona mediante long-polling ligero: consulta `getUpdates` cada segundo para recibir pulsaciones de botones (`callback_query`) y mensajes de texto.

Componentes principales

Componente | Función

```

`HealthHandler` | Servidor HTTP en segundo plano para mantener activo el servicio en Koyeb/Render

`sync_yaml_to_db()` | Sincroniza las fuentes de `sources.yaml` a la base de datos Supabase al arrancar

`send_message()` | Envía un mensaje nuevo con teclado de botones opcional

`edit_message()` | Edita un mensaje existente (simula navegación sin crear mensajes nuevos)

`answer_callback()` | Responde al callback del botón (elimina el spinner de carga del botón)

`get_updates()` | Consulta nuevos eventos (mensajes y pulsaciones de botones) con short polling

`handle_callback()` | Enrutador principal: procesa cada botón pulsado y llama a la función correspondiente

`handle_text_message()` | Procesa mensajes de texto libre según el estado conversacional del usuario

`run_news_loop()` | Tarea en segundo plano que ejecuta `bot_v2.py` cada 30 minutos

```

Menús disponibles

- **Gestión de Fuentes:** Ver, agregar, eliminar y activar/desactivar fuentes
- **Curación de Noticias:** Ver y aprobar noticias pendientes en la BD
- **Videos YouTube:** Enviar últimos vídeos de canales al canal de Telegram
- **Configuración:** Cambiar modelo de IA, editar prompt del sistema

- **Historial:** Ver últimas noticias enviadas, buscar por palabra clave
 - **Estadísticas:** Total enviadas, tasa de aprobación, top fuentes
 - **Logs:** Últimos eventos del sistema
-

9. Documentación del Código — sources.yaml

```
rss_feeds:          # Lista de feeds RSS
- name: "Xataka"    # Nombre que aparece en los logs y en la noticia
  url: "https://www.xataka.com/feedburner.xml" # URL del feed RSS/Atom
  tags: ["tech"]    # Categoría (no afecta al filtrado automático)

youtube_channels:   # Lista de canales de YouTube
- name: "midudev"
  channel_id: "UC8LeXCW0alN8SxlrPcG-PaQ" # ID único del canal (en la URL)
```

Fuentes RSS activas (13):

GitHub Blog, CSS-Tricks, freeCodeCamp, El País Tecnología, Computer Hoy, Hipertextual, Xataka, Genbeta, Muycomputer, Computerworld España, The Verge, Wired

Canales YouTube activos (13):

Andrej Karpathy, Two Minute Papers, Yannic Kilcher, Lex Fridman, AI Explained, Matt Wolfe, DotCSV, midudev, MoureDev, Fireship, Theo (t3.gg), Veritasium

10. Documentación del Código — GitHub Actions

```
name: ■ Tech Digest Bot – Daily Digest

on:
  schedule:
    # Ejecuta cada hora de 6:00 a 21:00 UTC
    # = 8:00 a 23:00 hora Madrid (verano, UTC+2)
    - cron: "0 6-21 * * *"

  workflow_dispatch: # Permite ejecución manual desde la web de GitHub
    inputs:
      bot_version: ...

jobs:
  send-digest:
    runs-on: ubuntu-latest # Máquina virtual Linux gratuita
    timeout-minutes: 10    # Aborta si tarda más de 10 minutos

    steps:
      # 1. Descarga el código del repositorio
      - uses: actions/checkout@v4

      # 2. Instala Python 3.11 con caché de pip (acelera ejecuciones futuras)
      - uses: actions/setup-python@v5
        with:
```

```

python-version: "3.11"
cache: "pip"

# 3. Instala las librerías de requirements.txt
- run: pip install -r requirements.txt -q

# 4. Ejecuta el bot con las credenciales de GitHub Secrets
- env:
  GROQ_API_KEY: ${ secrets.GROQ_API_KEY }
  TELEGRAM_BOT_TOKEN: ${ secrets.TELEGRAM_BOT_TOKEN }
  TELEGRAM_CHAT_ID: ${ secrets.TELEGRAM_CHAT_ID }
  run: python bot_v2.py

# 5. Muestra resumen aunque el paso anterior falle (if: always())
- if: always()
  run: echo "■ Workflow completado"

```

Funcionamiento: GitHub ejecuta este workflow en una máquina virtual Ubuntu limpia cada hora dentro del rango configurado. La máquina descarga el código, instala dependencias y ejecuta el bot. Al terminar, la máquina se destruye. No hay servidor permanente.

11. Variables de Entorno y Secrets

Variable | Dónde se usa | Cómo obtenerla

`GROQ\_API\_KEY` | `bot\_v2.py` — llamadas a LLaMA | console.groq.com → API Keys

`ANTHROPIC\_API\_KEY` | Subagentes manuales en Claude Code | console.anthropic.com

`TELEGRAM\_BOT\_TOKEN` | Todos los scripts — autenticación del bot | @BotFather en Telegram

`TELEGRAM\_CHAT\_ID` | Todos los scripts — canal destino | ID numérico del canal (negativo)

`SUPABASE\_URL` | `db\_manager.py` — conexión BD | Proyecto en supabase.com

`SUPABASE\_KEY` | `db\_manager.py` — autenticación BD | Proyecto en supabase.com → API

Local (.env): Todas las variables se guardan en el fichero .env para uso local.

GitHub Actions: Las variables GROQ_API_KEY, TELEGRAM_BOT_TOKEN y TELEGRAM_CHAT_ID se configuran en Settings → Secrets and variables → Actions del repositorio.

12. Costes del Sistema

Servicio | Plan | Uso mensual | Límite gratuito | Coste

Groq API | Free | ~4M tokens | 500k tokens/día | \$0

GitHub Actions | Free | ~960 min | 2.000 min/mes | \$0

Telegram Bot API | Free | Ilimitado | Ilimitado | \$0

Supabase | Free | ~75 registros | 500 MB | \$0

13. Instrucciones de Uso

Ejecución manual del bot

```
cd C:\Users\NitroPC\Desktop\Bot_News
python bot_v2.py
```

Arrancar el panel de control

```
cd C:\Users\NitroPC\Desktop\Bot_News
python telegram_bot.py
```

Luego abrir Telegram → buscar @IsMa_Noticias_bot → escribir /start.

Añadir una nueva fuente RSS

Editar sources.yaml y añadir bajo rss_feeds:

```
- name: "Nombre de la Fuente"
  url: "https://ejemplo.com/feed.rss"
  tags: ["tech"]
```

Añadir un canal de YouTube

Editar sources.yaml y añadir bajo youtube_channels:

```
- name: "Nombre del Canal"
  channel_id: "UCxxxxxxxxxxxxxxxxxxxxxxxxxx"
```

El channel_id se encuentra en la URL del canal de YouTube.

Forzar ejecución manual en GitHub

Ir a GitHub → Repositorio → Actions → Tech Digest Bot → Run workflow.

Ajuste de horario en invierno (octubre-marzo)

Cambiar en .github/workflows/daily.yml:

```
- cron: "0 7-22 * * *" # 8:00-23:00 Madrid invierno (UTC+1)
```